

ADP470S: Algorithmic Analysis and Data Structures

This document serves as a comprehensive study resource for ADP470S, covering key concepts in algorithmic analysis, recursion vs. iteration, data structures, and sorting algorithms.

SO1: Algorithm Analysis and Complexity Metrics

Primary Metrics

- Step Count:** The total number of primitive operations (e.g., additions, assignments, comparisons) an algorithm performs. This metric is often analyzed using Big O notation to describe its growth rate.
- Extra Memory:** The additional memory an algorithm consumes beyond the space required for the input data. For recursive algorithms, this is primarily determined by the runtime call stack.

Algorithm Comparison: Recursive vs. Iterative

Metric	Recursive Approach	Iterative Approach
Step Count	Grows with recursion depth; each call is constant work.	Typically linear ($O(n)$) relative to input n .
Extra Memory	Linear ($O(n)$): One stack frame per recursive call.	Constant ($O(1)$): Reuses memory.

The Runtime Call Stack

Recursive algorithms use the runtime call stack to manage memory. Each function call pushes a new stack frame onto the stack, containing parameters, local variables, and the return address. Iterative solutions avoid this overhead by using a single stack frame.

A **Stack Overflow Error** occurs when the call stack runs out of memory, often due to infinite recursion (missing base case) or excessive recursion depth.

SO2: Recursive vs. Iterative Computations

Core Definitions

- Recursion:** A technique where a method calls itself with a reduced argument. It requires a **Base Case** (stopping condition) and a **Recursive Case** (logic that moves toward the base case).
- Iteration:** A technique using loop structures (e.g., for, while) to repeat logic until a condition is met, without additional method calls.

Case Study 1: Factorial ($n!$)

Mathematical Definition: $n! = n \times (n - 1)!$, with the base case $0! = 1$.

Recursive Java Implementation:

```
public static int factorial(int n) {
    if (n == 0) { // Base case
        return 1;
    }
    return n * factorial(n - 1); // Recursive case
}
```

Iterative Java Implementation:

```
        result *= i;
    }
    return result;
}
```

Call Expansion and Unwinding Trace for factorial(5):

- factorial(5) calls factorial(4)
- factorial(4) calls factorial(3)
- factorial(3) calls factorial(2)
- factorial(2) calls factorial(1)
- factorial(1) calls factorial(0)
- Base Case Reached:** factorial(0) returns 1.
- Unwinding:** factorial(1) returns $1 \times 1 = 1$.
- Unwinding:** factorial(2) returns $2 \times 1 = 2$.
- Unwinding:** factorial(3) returns $3 \times 2 = 6$.
- Unwinding:** factorial(4) returns $4 \times 6 = 24$.
- Final Result:** factorial(5) returns $5 \times 24 = 120$.

Case Study 2: Fibonacci Sequence

Mathematical Definition: $F(n) = F(n - 1) + F(n - 2)$ for $n \geq 2$, with base cases $F(0) = 1$ and $F(1) = 1$. (Note: This follows the source context where the sequence starts 1, 1, 2, 3, 5, 8...).

Recursive Java Implementation:

```
public static int fibonacci(int n) {
    if (n <= 1) {
        return 1; // Base case reflecting F(0)=1, F(1)=1
    }
    return fibonacci(n - 1) + fibonacci(n - 2);
}
```

Recursive Call Tree & Inefficiency: The naïve recursive approach has exponential time complexity, $O(2^n)$, due to repeated computations (e.g., $F(3)$ is computed multiple times when calculating $F(5)$).

Iterative Java Implementation:

```
public static int fibonacci(int n) {
    if (n <= 1) {
        return 1;
    }
    int prev = 1, curr = 1;
    for (int i = 2; i <= n; i++) {
        int next = prev + curr;
        prev = curr;
        curr = next;
    }
    return curr;
}
```

Complexity Comparison Table

Algorithm	Approach	Time Complexity	Space Complexity
Factorial	Recursive	$O(n)$	$O(n)$
Factorial	Iterative	$O(n)$	$O(1)$
Fibonacci	Recursive	$O(2^n)$	$O(n)$
Fibonacci	Iterative	$O(n)$	$O(1)$

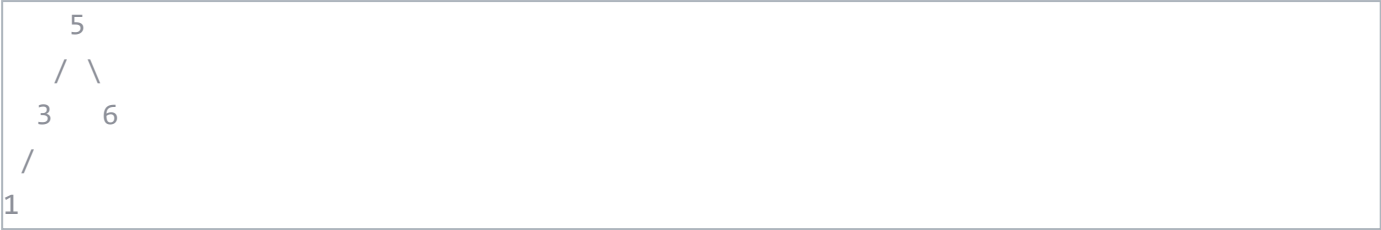
SO3: Search Algorithms & BST Properties

Binary Search Tree (BST) Properties

- Structure:** Each node has at most two children.

- **Uniqueness:** Duplicate values are not allowed.

Visual Hierarchy Example (Root = 5):



The value 1 is the left child of 3 because $1 < 3$. Both 1 and 3 are in the left subtree of 5 because $1 < 5$ and $3 < 5$.

SO6: Recursive Data Structures & Linked Lists

A **Linked List** can be defined recursively: it consists of a **Head** element and a **Sub-list** (the tail), which is itself a list.

Example Sequence: $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 7 \rightarrow 8 \rightarrow 9 \rightarrow 10$. (Note: The number 6 is skipped in this specific sequence).

Summary of Comparison Metrics

Readability vs. Performance

- **Readability:** Recursive solutions are often favored for their conciseness and close resemblance to mathematical definitions, making them easier to understand and maintain.
- **Performance:** Iterative solutions are generally preferred in production due to better performance. They avoid the overhead of function calls (stack frame management) and typically achieve constant space complexity ($O(1)$).

Essential Checklist for Recursive Functions

Every recursive function must include:

1. **Base Case:** A defined stopping condition to prevent infinite recursion.
2. **Recursive Case:** A step that reduces the problem size and moves closer to the base case.

SO4 & SO5: Bubble Sort and 2D Arrays

- **Bubble Sort:** A simple sorting algorithm that repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order. Its time complexity is typically $O(n^2)$.
- **2D Arrays:** Technical implementation details and specific limitations are often covered in secondary materials.

Complexity Classes (Big O Notation)

Big O notation describes the worst-case performance of an algorithm as the input size (n) grows.

- $O(1)$: **Constant Time** – Execution time is independent of input size.
- $O(\log n)$: **Logarithmic Time** – Execution time grows very slowly; the problem size is halved at each step (e.g., Binary Search).
- $O(n)$: **Linear Time** – Execution time grows directly proportional to input size.
- $O(n^2)$: **Quadratic Time** – Execution time grows rapidly; often seen with nested loops (e.g., Bubble Sort).
- $O(2^n)$: **Exponential Time** – Execution time grows extremely fast; considered inefficient for larger inputs (e.g., naïve recursive Fibonacci).

Searching Algorithms

Linear Search

- **Definition:** Iterates through a collection one element at a time.
- **Complexity:**
 - Best Case: $O(1)$ (element found at the first position).

Binary Search

- **Definition:** Requires a sorted data structure. Repeatedly divides the search interval in half.
- **Complexity:** $O(\log n)$.

Sorting Algorithms

Bubble Sort

- **How it works:** Compares adjacent elements and swaps them if they are out of order. Repeats passes until no swaps are needed.
- **Time Complexity:** $O(n^2)$ (worst and average case). Can be $O(n)$ if the list is already sorted (with an optimization to detect no swaps).
- **Space Complexity:** $O(1)$ (in-place sort).

Selection Sort

- **How it works:** Finds the minimum element from the unsorted part and puts it at the beginning.
- **Time Complexity:** $O(n^2)$.
- **Space Complexity:** $O(1)$.

Insertion Sort

- **How it works:** Builds the final sorted array one item at a time. Takes an element from the input data and finds its correct position within the already sorted portion.
- **Time Complexity:** $O(n^2)$ (worst and average case). $O(n)$ in the best case (already sorted).
- **Space Complexity:** $O(1)$.

Quick Sort

- **How it works:** A "divide and conquer" algorithm. It picks a 'pivot' element and partitions the other elements into two sub-arrays according to whether they are less than or greater than the pivot. It then recursively sorts the sub-arrays.
- **Time Complexity:**
 - Best Case: $O(n \log n)$
 - Average Case: $O(n \log n)$
 - Worst Case: $O(n^2)$ (occurs with poor pivot selection, e.g., on already sorted data).
- **Space Complexity:** $O(\log n)$ on average (due to recursion stack), $O(n)$ in the worst case.

Merge Sort

- **How it works:** Another "divide and conquer" algorithm. It divides the input array into two halves, calls itself for the two halves, and then merges the two sorted halves.



Materials



Study Guide



Smart Study



Flashcards



Test